

# **Module 9: Records (optional)**

# Overview

- Records
- Mutability
- Value Equality
- Inheritance
- Deconstruction

# Record Reference Type

>= C# 9.0

- Simple syntax for creating a reference type with immutable properties

```
record Employee(string FirstName, string LastName);
```

SYNTACTIC SUGAR

```
Employee e = new Employee("Joe", "Doe");  
e.FirstName = "John";    // Error: init-Only property
```

- Generated Class with Ctor and Init-only properties

```
class Employee  
{  
    public string FirstName { get; init; }  
    public string LastName { get; init; }  
  
    public Employee(string firstName, string lastName)  
    {  
        FirstName = firstName;  
        LastName = lastName;  
    }  
}
```

# Lab 09.1 - Records

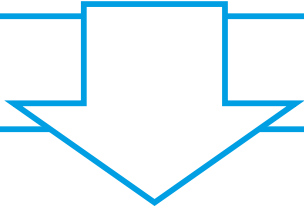


Exercise:

- Using Records

# Custom Init-only properties

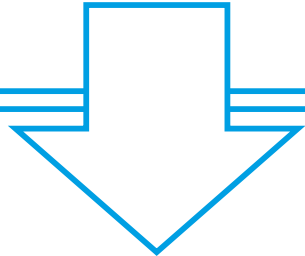
```
record Employee(string FirstName, string LastName);
```



- You can also write your own properties

```
record Employee
{
    public string FirstName { get; init; } = default!;
    public string LastName { get; init; } = default!;
};
```

```
Employee e = new Employee();
e.FirstName = "John"; // Error: init-Only property
```



```
class Employee
{
    public string FirstName { get; init; }
    public string LastName { get; init; }

    public Employee() // Generated default ctor
    {
        FirstName = null;
        LastName = null;
    }
}
```

# Custom properties can be Mutable

- Mutable properties with setter

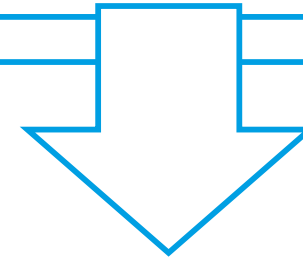
```
record Employee
{
    public string FirstName { get; set; } = default!;
    public string LastName { get; set; } = default!;
};
```

SYNTACTIC SUGAR

```
Employee e = new Employee();
e.FirstName = "John"; // OK
```

```
class Employee
{
    public string FirstName { get; set; }
    public string LastName { get; set; }

    public Employee() // Generated default ctor
    {
        FirstName = null;
        LastName = null;
    }
}
```



# Custom + auto-implemented prop

You can combine

- generated auto-implemented property definition
- your own property definition

```
record Employee(string FirstName, string LastName, string Department)
{
    public string Department { get; set; } = default!;
}
```

```
Employee e = new Employee("Joe", "Doe", "IT");
e.FirstName = "John";           // Error: init-Only property
e.Department = "Finance";      // OK
```

*Note:*

- *The main purpose of Records is to generate immutable types*

# The main purpose of Records

- Mostly Support for Immutable Data Models  
(Simple basic syntax for generating reference / value types)
- Additional capabilities of Records:
  - Ability to compare value (Value equality)
  - Simple basic syntax for nondestructive mutation
  - Simple conversion to String
  - Inheritance support
  - Ability to Deconstruct



# Ability to compare value (Value equality)

```
record Employee(string FirstName, string LastName);
```

```
Console.WriteLine(Object.ReferenceEquals(e1, e2)); // false
Console.WriteLine(e1.Equals(e2));                 // true
Console.WriteLine(e1 == e2);                       // true
```

- Overriden Equals, GetHashCode and Overloaded == and !=

```
public override bool Equals(Employee other)
{
    return (object)this == other || EqualityContract == other.EqualityContract;
}

public static bool operator ==(Employee left, Employee right)
{
    return (object)left == right || ((object)left != null && left.Equals(right));
}

public static bool operator !=(Employee left, Employee right)
{
    return !(left == right);
}

public override int GetHashCode()
{
    return (EqualityComparer<Type>.Default.GetHashCode(EqualityContract) );
}
```

# Nondestructive Mutation

```
record Employee(string FirstName, string LastName);
```

- Copy of reference

```
Employee e1 = new Employee("Joe", "Doe");  
Employee e2 = e1;  
Console.WriteLine(Object.ReferenceEquals(e1, e2)); // true,
```

- Copy of object

```
Employee e1 = new Employee("Joe", "Doe");  
Employee e2 = e1 with { FirstName = "John"};  
Console.WriteLine(Object.ReferenceEquals(e1, e2)); // false
```

SYNTACTIC SUGAR



```
Employee e1 = new Employee("Joe", "Doe");  
Employee e2 = employee.<Clone>$();  
e2.FirstName = "John";
```

# Inheritance Support

- A record can inherit from another record

```
abstract record Person(string FirstName, string LastName);  
  
record Employee(string FirstName, string LastName, string Department)  
    : Person(FirstName, LastName);
```

- Record can't inherit from a class

```
class Person { }  
record Employee() : Person;
```



- Class can't inherit from a record

```
record Person(string FirstName, string LastName);  
class Employee() : Person{}
```



# Simple conversion to String

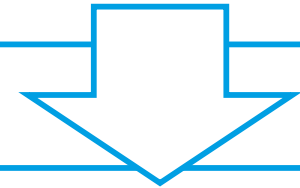
- Record types have a compiler-generated ToString method

```
public override string ToString()
{
    StringBuilder stringBuilder = new StringBuilder();
    stringBuilder.Append("Employee");
    stringBuilder.Append(" { ");
    if (PrintMembers(stringBuilder))
    {
        stringBuilder.Append(" ");
    }
    stringBuilder.Append("}");
    return stringBuilder.ToString();
}
```

- Tip: Compiler generates PrintMembers method as virtual, it can be overridden to implement custom formatting*

# Ability to Deconstruct

```
record Employee(string FirstName, string LastName);
```



```
public void Deconstruct(out string FirstName, out string LastName)
{
    FirstName = base.FirstName;
    LastName = base.LastName;
}
```

- Deconstruct method generated for all positional properties

```
Person person = new Person("Joe", "Doe");
var (fname, lname) = person;           // Deconstruction
Console.WriteLine($"{fname} {lname}");
```

# Review

- Records
- Mutability
- Value Equality
- Inheritance
- Deconstruction